Università degli Studi di Verona

CORSO DI LAUREA IN INFORMATICA

Basi di dati

Federico Brutti
federico.brutti@studenti.univr.it

Inserire citazione inerente alla materia

Contents

1 Progettazione di una base di dati **3**
1.1 Applicazioni gestionali e DBMS 3
1.2 Creazione di una base di dati 4
1.3 Progettazione concettuale 5
1.4 Progettazione logica 6

2 Interazione con un database **7**

3 RDBMS e SQL in postgreSQL **8**

4 Query e transazioni **9**

5 Accesso a database da programmi Python **10**

6 Uso di DBMS, NoSQL e MongoDB **11**

7 Database Management Systems **12**

8 Interrogazioni in SQL **13**

9 Architettura NoSQL MongoDB **14**

Chapter 1

Progettazione di una base di dati

1.1 Applicazioni gestionali e DBMS

Le **Applicazioni** informatiche in ambito gestionale sono nate negli anni '70 e sono caratterizzate da algoritmi di elaborazione semplici e una grande quantità di dati condivisi da più applicazioni, ambo derivanti dall'ambiente in cui sono introdotte, chiamato **sistema informativo**. Quest'ultimo è definito formalmente come l'*insieme delle attività umane e dei dispositivi di memorizzazione ed elaborazione atte alla gestione e all'organizzazione di informazioni di interesse per un'organizzazione.* Il sistema informativo è composto a sua volta da due componenti:

- **Dato**: Elemento di conoscenza di base, costituito da simboli da elaborare.
- **Informazione**: Interpretazione dei dati che consente di ottenere conoscenza di fatti e situazioni.

È possibile studiare il comportamento dei sistemi informativi mediante diagrammi di flusso, i quali **definiscono** archivi, sorgenti, utenti, procedure e flussi di dati, quindi rappresentano le informazioni come collezioni di dati. Questi elementi portano a quella che è chiamata **soluzione convenzionale**, che mostra i vari programmi interagire col file system, il quale mantiene i dati. Tuttavia presenta alcuni problemi, come per esempio un accesso lento ai dati e la loro ridondanza.

Un'altra soluzione pone come middleware posto tra le applicazioni e il file system un **Database Management System** (DBMS), un sistema che gestisce le **basi di dati** su memoria secondaria, assicurando affidabilità, privatezza e accesso efficiente. Presenta inoltre due vantaggi principali:

- **Maggiore astrazione**: I dati sono visti come oggetti o righe di tabelle quando il DBMS si interfaccia con le applicazioni.
- **Maggiore efficacia**: L'accesso ai dati è basato su un linguaggio di interazione.

In particolare, i **linguaggi di interazione** si dividono in due categorie: la prima è atta alla **definizione dei dati** (DDL), mentre la seconda concerne la **manipolazione dei dati** (DML), a sua volta suddivisa in **linguaggi di interrogazione**, per estrarre informazioni dal DB, e **di manipolazione**, per popolare il DB o modificarne i dati. Il DBMS fornisce inoltre un **Modello dei Dati**, l'insieme dei costrutti per la descrizione di struttura e proprietà dell'informazione all'interno del DB. Ad oggi, vengono utilizzati i seguenti modelli:

- Modello relazionale: SQL92 o SQL2
- Modello a oggetti
- Modello object-relational: SQL99 o SQL3
- Modello basato su documenti: JSON
- Modelli NoSQL: Basati su colonne, costrutti chiave-valore

Bisogna tuttavia fare attenzione a distinguere il modello dei dati dallo **schema del DB**, la descrizione di struttura e proprietà della base di dati che fa uso dei costrutti del modello, e dall'**istanza del DB**, costituita dagli effettivi valori che popolano le strutture dati in una determinata istanza. In soldoni, il modello definisce i costrutti, lo schema la loro relativa implementazione e l'istanza una situazione specifica con determinati valori.

Passiamo ora all'architettura generale dei DBMS: presentano tre livelli: **esterno**, che descrive una porzione dello schema logico per un'interfaccia specifica, **logico**, la rappresentazione di struttura e proprietà del DB, definite con il modello dei dati, e **interno**, rappresentante il DB mediante le strutture fisiche di memorizzazione. In un contesto tale è necessario chiarire che le basi di dati devono mantenere due tipi di indipendenza:

- **Fisica**: Schema logico è indipendente da quello interno, facendo sì che la variazioni sul secondo non impattino il primo.
- **Logica**: Schemi esterni indipendenti da quello logico, facendo sì che le variazioni nel secondo, purché non rimuovano dati, non impattano i primi.

1.2 Creazione di una base di dati

Per progettare adeguatamente una base di dati è necessario studiare il ciclo di vita del processo di automazione all'interno di un sistema informativo. In primo luogo è richiesto uno **studio di fattibilità** per definire costi ed eventuali alternative; segue la **raccolta e analisi dei requisiti** del sistema, producendo una descrizione completa ma informale, per poi terminare con l'effettiva **progettazione**. Quest'ultima fase è suddivisa in due attività:

- **Progettazione dei dati**: Descrizione formale dei dati attraverso lo schema, comprende l'implementazione del DBMS seguita dalla validazione e collaudo.
- **Progettazione delle applicazioni**: Descrizione formale delle applicazioni attraverso le specifiche.

In questo corso ci concentreremo sulle metodologie di progettazione dei dati. Un buon metodo è costituito da una decomposizione iniziale in passi dell'attività di progetto, attuazione di strategie e modelli di riferimento da seguire, come anche di criteri di scelta da applicare. La metodologia deve inoltre essere generale, semplice da usare, ma soprattutto in grado di produrre un risultato completo e corretto.

Nel concreto, la progettazione dei dati presenta tre fasi principali: **concettuale**, per rappresentare il contenuto informativo del DB formalmente e indipendente da tecnologia e operazioni, portando alla creazione dello schema concettuale, **logica** per tradurre lo schema concettuale in quello logico, aderente al modello dei dati del DBMS scelto, e **fisica**, per completare lo schema logico coi parametri relativi alla memorizzazione fisica dei dati e metodi di accesso per garantire efficienza, fornendo lo schema fisico.

1.3 Progettazione concettuale

Il modello storicamente usato per la progettazione concettuale delle basi di dati è quello **Entità-Relazione** (ER); propone un insieme di strumenti grafici formali e indipendenti dalla tecnologia[1], detti **costrutti**, per specificare struttura e proprietà dei dati da rappresentare al suo interno.

La definizione di ogni costrutto si ha specificando il loro significato, sintassi grafica e relative istanze; quelli di base sono:

- **Entità**: Classe di oggetti con attributi comuni, esistenza autonoma e identificazione univoca, rappresentate con un rettangolo contenente il loro nome.
- **Relazione**: Legame logico tra due o più entità, rappresentate da un rombo a cui si collegano attraverso linee le entità coinvolte. Il nome è posto vicino alla forma geometrica.
- **Attributo**: Proprietà elementare di un'entità o una relazione; associa a ogni loro istanza un singolo valore appartenente a un dominio, indicandolo con una piccola riga alla cui estremità è posto un pallino vuoto con a fianco il suo nome.

Ognuno di questi costrutti si può **istanziare**, ovvero concretizzare, all'interno della base di dati; sono annotati con $I(nomeCostrutto)$ e i relativi insiemi sono detti **popolazioni**.

[1] Privi di qualsivoglia ottimizzazioni o semplificazioni che possono avere luogo in base agli strumenti scelti.

Alla nascita della base di dati ogni popolazione è vuota, poiché nessun costrutto è stato ancora istanziato. Istanziare un'entità significa aggiungere un oggetto alla base di dati e farlo per un attributo corrisponde ad associare un valore del dominio che assume a un'istanza di entità o relazione. Per queste ultime in particolare, definiamo le loro istanze come n-uple di istanze di entità; formalmente:

$$(e_1, ..., e_n), \text{ con } e_i \in I(E_i), 1 \leq i \leq n$$

E la relativa popolazione rappresenta l'insieme delle coppie delle due entità coinvolte relazionate in un certo istante di tempo. Si tratta di una rappresentazione univoca, quindi non è possibile annotare più n-uple allo stesso modo.

1.4 Progettazione logica

Chapter 2

Interazione con un database

Chapter 3

RDBMS e SQL in postgreSQL

Chapter 4

Query e transazioni

Chapter 5

Accesso a database da programmi Python

Chapter 6

Uso di DBMS, NoSQL e MongoDB

Chapter 7

Database Management Systems

Chapter 8

Interrogazioni in SQL

Chapter 9

Architettura NoSQL MongoDB